Sornam AI — Voice Coverage Handover

Which backend actions Pratham and Amaan each convert into voice-callable tools

Companion to the ownership split (v1.0, 30 Jun 2026) · Backend: NestJS modular monolith · Pipeline reference: `apps/api/docs/VOICE_PIPELINE.md`

What "voice-callable" means here. A spoken command runs through one pipeline — `STT → router (picks one action) → resolver (names→IDs) → preview (validate + spoken read-back) → confirm gate → command bus → your service`. Making an endpoint voice-callable is *not* new NLU code: you register the action in the catalog and add one dispatcher case that calls your existing module service. The router, confirm gate and slot-filling are already generic.

The recipe — how each owner adds one voice action

1. **Catalog entry** (in your own `*.voice-actions.ts` registry — see prerequisite below): action name, the **execution Zod schema** (reuse your existing module schema), and metadata — `description`, `requiresConfirmation`, `sensitiveFields`, `argumentGuide` (the natural-language hint the LLM router reads), and `resolves` (which entities the resolver must look up).
2. **Dispatcher case** in `voice-command-bus.service.ts` → call your module's public service method. Mechanical, one line.
3. **Resolver** (only if the command references an entity by name): add the entity to `VoiceEntityRef` and a lookup in `VoiceResolverService` so "Ravi's job" becomes a UUID, with a clarifying question when ambiguous.
4. **Read-back** (optional): a spoken confirmation line in `voice-preview.service.ts`; otherwise a generic one is used.
5. **Test**: a resolver unit test (mocked Prisma) and, if you extend the offline router, a deterministic-router test.

Shared prerequisite (do first, together with Ayush). Today the catalog and dispatcher are single files in Ayush's voice module — if you both edit them you get constant merge conflicts, which breaks the "nobody edits shared files" rule. Split the catalog into **per-owner registries** (`relationship.voice-actions.ts` for Pratham, `ops.voice-actions.ts` for Amaan) that the voice module composes. Then each of you works inside your own boundary. New resolver entities are the only shared touch-point: Pratham adds `followup`; Amaan adds `inventory_item`, `karigar`, `karigar_job`.

WRITE mutates data → always confirm-gated (spoken read-back before save). READ answers a question → runs immediately. P1 showcase / high value P2 useful P3 nice-to-have

Already voice-callable (do not re-add): Pratham — `create_customer`, `create_repair_order`, `update_repair_status`, `create_scheme`, `record_scheme_installment`. Amaan — `stock_summary`, `create_buyback_item`.

Pratham — M2 Relationship / WhatsApp / Schemes & Repairs

Modules: `customers`, `repairs`, `schemes`, `whatsapp`. Registry to own: `relationship.voice-actions.ts`. New resolver entity: `followup`.

ENDPOINT / SERVICE	VOICE ACTION	TYPE	RESOLVE	PRI
`GET /customers/follow-ups/list`	`list_pending_followups` "who should I win back / follow up?"	READ	—	P1
`GET /customers/:id/summary`	`customer_summary` "tell me about Lakshmi"	READ	customer	P1
`GET /repairs` + `/summary`	`repairs_status_report` "which repairs are ready?"	READ	—	P1
`GET /schemes/summary`	`schemes_due_report` "which schemes mature this month?"	READ	—	P1
`POST /customers/follow-ups`	`create_followup`	WRITE	customer	P2
`PATCH /customers/:id`	`update_customer` "update Lakshmi's phone / opt-in"	WRITE	customer	P2
`POST /customers/wholesale/orders`	`create_distributor_order`	WRITE	customer	P2
`PATCH /schemes/:id/status`	`update_scheme_status` (close / mature)	WRITE	scheme	P3
`PATCH /customers/follow-ups/:id/status`	`update_followup_status`	WRITE	followup	P3
`POST /whatsapp/send-request` **(not built)**	`send_whatsapp_message` opt-in checked	WRITE	customer	P1 ⚠

⚠ **Blocked:** WhatsApp send is an unimplemented interface (`WhatsAppNotImplemented`). Wire the provider first; the voice action is trivial after that.

Pratham also adds: resolver lookups for `customer` already exist; add `followup` resolution (by customer + latest open). Read-back templates for the write actions (customer/scheme/followup) in the preview service.

Amaan — M3 Stock / M5 Karigar / M4 Content / Buyback

Modules: `inventory`, `content`, `karigar`, `buyback`. Registry to own: `ops.voice-actions.ts`. New resolver entities: `inventory_item`, `karigar`, `karigar_job`.

ENDPOINT / SERVICE	VOICE ACTION	TYPE	RESOLVE	PRI
`GET /inventory/slow-stock`	`slow_stock_report` "what stock is stuck?"	READ	—	P1
`POST /content/slow-stock/promote`	`promote_slow_stock` "promote slow movers"	WRITE	—	P1
`POST /karigars/jobs`	`issue_karigar_job` "issue 50g to Ravi"	WRITE	karigar	P1
`POST /karigars/jobs/:id/returns`	`record_karigar_return` wastage calc	WRITE	karigar job	P1
`GET /karigars/:id/scorecard`	`karigar_scorecard` "how is Ravi doing?"	READ	karigar	P2
`POST /inventory/items`	`create_inventory_item` (voice stock intake)	WRITE	—	P2
`POST /inventory/movements`	`record_stock_movement` (adjustment)	WRITE	inventory item	P2
`POST /content/requests`	`create_content_request` "make a post for this item"	WRITE	inventory item	P2
`GET /buyback/summary`	`buyback_summary`	READ	—	P2
`POST /buyback/bundles` + `/:id/items`	`create_buyback_bundle` / `assign_buyback_items`	WRITE	—	P2
`PATCH /inventory/items/:id/status`	`update_inventory_status` (reserve / sold)	WRITE	inventory item	P3
`POST /karigars`	`create_karigar`	WRITE	—	P3

⚠ **Partial:** content generation is an unimplemented interface (`ContentNotImplemented`). `promote_slow_stock` and `create_content_request` create *records* and work by voice today, but no image/reel is produced until the provider is wired.

Amaan also adds: resolver lookups for `inventory_item` (by SKU/name), `karigar` (by name), `karigar_job` (latest open job for a karigar). Read-back templates for stock/karigar/content writes — the karigar-return read-back should speak the computed wastage before saving.

Definition of done (per action)

- Action appears in `GET /voice/sessions/actions` and is selected by both routers (deterministic optional, Sarvam LLM required).
- Every WRITE reads its resolved values back and writes nothing before a spoken "yes"; every READ answers immediately.
- Entity references resolve by name, and ask a clarifying question when ambiguous or missing.
- Args validate against the module's existing Zod schema (generalized slot-filling asks for anything missing).
- Resolver unit test passes; typecheck and the voice suite stay green.

Explicitly out of scope for voice (do not convert)

Browse/list endpoints, CSV import & export, asset-upload callbacks, and admin/access routes. Voice is for quick actions and questions, not table paging. `metal-rates` stays with Ayush (shared commerce).